

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО–КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №10
дисциплины
«Основы нейронных сетей»
Вариант 10

Выполнил:
Рябинин Егор Алексеевич
3 курс, группа ИВТ-б-о-23-2,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Проверил:
Доцент департамента цифровых,
робототехнических систем и
электроники института перспективной
инженерии
Воронкин Роман Александрович

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г

Тема: сравнительный анализ архитектур глубоких нейронных сетей для сегментации изображений.

Цель: изучить современные архитектуры глубоких нейронных сетей для задачи семантической сегментации изображений, провести сравнительный анализ их эффективности, исследовать влияние гиперпараметров на качество обучения и освоить методы оптимизации вычислительных ресурсов при работе с большими наборами данных.

Порядок выполнения работы:

Ссылка на репозиторий:

https://github.com/bohemiaaaaa/Lab10_Neural-networks

Pytorch:

Задание 1. На основе учебного ноутбука проведите финальную подготовку данных. Измените количество сегментирующих классов с 16 на 5. Проведите суммарно не менее 10 экспериментов и визуализируйте их результаты (включая точность обучения сетей на одинаковом количестве эпох, например, на 7), изменив:

- количество фильтров в сверточных слоях
- размер ядра в сверточных слоях
- функцию активации в скрытых слоях

```

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as transforms

import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
import os
import time
import gc
import gdown

from sklearn.model_selection import train_test_split
from tensorflow.keras.preprocessing import image

print(f"PyTorch version: {torch.__version__}")
print(f"CUDA available: {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")

```

Рисунок 1 – Импорт библиотек

```

# Загрузка датасета из облака
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/l14/construction_256x192.zip', None, quiet=False)
!unzip -q 'construction_256x192.zip'

```

Рисунок 2 – Загрузка датасета

```

# Глобальные параметры

IMG_WIDTH = 192          # Ширина картинки
IMG_HEIGHT = 256         # Высота картинки
NUM_CLASSES = 16         # Задаем количество классов на изображении
TRAIN_DIRECTORY = 'train' # Название папки с файлами обучающей выборки
VAL_DIRECTORY = 'val'     # Название папки с файлами проверочной выборки

```

Рисунок 3 – Глобальные параметры

```

train_images = []
val_images = []

cur_time = time.time()

for filename in sorted(os.listdir(TRAIN_DIRECTORY+'/original')):
    train_images.append(image.load_img(os.path.join(TRAIN_DIRECTORY+'/original', filename),
                                                target_size=(IMG_WIDTH, IMG_HEIGHT)))

print('Обучающая выборка загружена. Время: ', round(time.time() - cur_time, 2), 'c')
print('Количество: ', len(train_images))

cur_time = time.time()

for filename in sorted(os.listdir(VAL_DIRECTORY+'/original')):
    val_images.append(image.load_img(os.path.join(VAL_DIRECTORY+'/original', filename),
                                                target_size=(IMG_WIDTH, IMG_HEIGHT)))

print('Проверочная выборка загружена. Время: ', round(time.time() - cur_time, 2), 'c')
print('Количество: ', len(val_images))

```

Рисунок 4 – Загрузка оригинальных изображений

```

train_segments = []
val_segments = []

cur_time = time.time()

for filename in sorted(os.listdir(TRAIN_DIRECTORY+'/segment')):
    train_segments.append(image.load_img(os.path.join(TRAIN_DIRECTORY+'/segment', filename),
                                                target_size=(IMG_WIDTH, IMG_HEIGHT)))

print('Обучающие маски загружены. Время: ', round(time.time() - cur_time, 2), 'c')
print('Количество: ', len(train_segments))

cur_time = time.time()

for filename in sorted(os.listdir(VAL_DIRECTORY+'/segment')):
    val_segments.append(image.load_img(os.path.join(VAL_DIRECTORY+'/segment', filename),
                                                target_size=(IMG_WIDTH, IMG_HEIGHT)))

print('Проверочные маски загружены. Время: ', round(time.time() - cur_time, 2), 'c')
print('Количество: ', len(val_segments))

```

Рисунок 5 – Загрузка сегментированных изображений

```

# Цвета для 5 классов
CLASS_COLORS_5 = [
    (0, 0, 0),      # фон
    (100, 0, 0),    # класс 1
    (0, 100, 0),    # класс 2
    (0, 0, 100),    # класс 3
    (100, 100, 0),  # класс 4
]
NUM_CLASSES = 5

def segments_to_class_map(seg_list, class_colors, num_classes=5):
    h, w = seg_list[0].size[1], seg_list[0].size[0]
    result = np.zeros((len(seg_list), h, w), dtype=np.int64)

    for i, seg in enumerate(seg_list):
        seg_arr = np.array(seg)
        for class_idx, color in enumerate(class_colors):
            mask = np.all(seg_arr == color, axis=-1)
            result[i, mask] = class_idx
    return result

# X как float32 нормализованные [0,1]
x_train = np.array([np.array(img, dtype=np.float32) / 255.0 for img in train_images])
x_val = np.array([np.array(img, dtype=np.float32) / 255.0 for img in val_images])

# Y как индексы классов
y_train = segments_to_class_map(train_segments, CLASS_COLORS_5, NUM_CLASSES)
y_val = segments_to_class_map(val_segments, CLASS_COLORS_5, NUM_CLASSES)

x_train = np.transpose(x_train, (0, 3, 1, 2))
x_val = np.transpose(x_val, (0, 3, 1, 2))

print(f"x_train: {x_train.shape} dtype={x_train.dtype}")
print(f"y_train: {y_train.shape} dtype={y_train.dtype}")

del train_images, val_images, train_segments, val_segments
gc.collect()

```

Рисунок 6 – Цветовая карта для 5 классов и подготовка данных

```

class SegmentationDataset(Dataset):
    def __init__(self, images, masks):
        self.images = torch.tensor(images, dtype=torch.float32)
        self.masks = torch.tensor(masks, dtype=torch.long)

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        return self.images[idx], self.masks[idx]

# Создаем датасеты
train_dataset = SegmentationDataset(x_train, y_train)
val_dataset = SegmentationDataset(x_val, y_val)

# DataLoader'ы
BATCH_SIZE = 16
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, num_workers=2, pin_memory=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=2, pin_memory=True)

print(f"Train batches: {len(train_loader)}, Val batches: {len(val_loader)}")

```

Рисунок 7 – Dataset и Dataloader

```

class DoubleConv(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size=3, activation='relu'):
        super(DoubleConv, self).__init__()

        if activation == 'relu':
            act = nn.ReLU(inplace=True)
        elif activation == 'elu':
            act = nn.ELU(inplace=True)
        elif activation == 'selu':
            act = nn.SELU(inplace=True)
        elif activation == 'linear':
            act = nn.Identity()
        else:
            act = nn.ReLU(inplace=True)

        self.double_conv = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size, padding=kernel_size//2, bias=False),
            nn.BatchNorm2d(out_channels),
            act,
            nn.Conv2d(out_channels, out_channels, kernel_size, padding=kernel_size//2, bias=False),
            nn.BatchNorm2d(out_channels),
            act
        )

    def forward(self, x):
        return self.double_conv(x)

class Down(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size=3, activation='relu'):
        super(Down, self).__init__()
        self.maxpool_conv = nn.Sequential(
            nn.MaxPool2d(2),
            DoubleConv(in_channels, out_channels, kernel_size, activation)
        )

    def forward(self, x):
        return self.maxpool_conv(x)

```

Рисунок 8 – U-net архитектура

```

class Up(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size=3, activation='relu'):
        super(Up, self).__init__()

        self.up = nn.ConvTranspose2d(in_channels, in_channels // 2, kernel_size=2, stride=2)
        self.conv = DoubleConv(in_channels, out_channels, kernel_size, activation)

    def forward(self, x1, x2):
        x1 = self.up(x1)
        diffY = x2.size()[2] - x1.size()[2]
        diffX = x2.size()[3] - x1.size()[3]
        x1 = nn.functional.pad(x1, [diffX // 2, diffX - diffX // 2,
                                     diffY // 2, diffY - diffY // 2])
        x = torch.cat([x2, x1], dim=1)
        return self.conv(x)

class UNet(nn.Module):
    def __init__(self, n_channels=3, n_classes=5, filters_base=24, kernel_size=3, activation='relu'):
        super(UNet, self).__init__()

        self.inc = DoubleConv(n_channels, filters_base, kernel_size, activation)
        self.down1 = Down(filters_base, filters_base*2, kernel_size, activation)
        self.down2 = Down(filters_base*2, filters_base*4, kernel_size, activation)
        self.down3 = Down(filters_base*4, filters_base*8, kernel_size, activation)
        self.down4 = Down(filters_base*8, filters_base*16, kernel_size, activation)

        self.up1 = Up(filters_base*16, filters_base*8, kernel_size, activation)
        self.up2 = Up(filters_base*8, filters_base*4, kernel_size, activation)
        self.up3 = Up(filters_base*4, filters_base*2, kernel_size, activation)
        self.up4 = Up(filters_base*2, filters_base, kernel_size, activation)

        self.outc = nn.Conv2d(filters_base, n_classes, kernel_size=1)

    def forward(self, x):
        x1 = self.inc(x)
        x2 = self.down1(x1)
        x3 = self.down2(x2)
        x4 = self.down3(x3)
        x5 = self.down4(x4)

        x = self.up1(x5, x4)
        x = self.up2(x, x3)
        x = self.up3(x, x2)
        x = self.up4(x, x1)

```

Рисунок 9 – U-net архитектура

```

def iou_score(pred, target, num_classes=5):
    ious = []
    pred = torch.argmax(pred, dim=1) # (B, H, W)

    for cls in range(num_classes):
        pred_mask = (pred == cls)
        target_mask = (target == cls)

        intersection = (pred_mask & target_mask).sum().float()
        union = (pred_mask | target_mask).sum().float()

        if union == 0:
            ious.append(float('nan'))
        else:
            ious.append((intersection / (union + 1e-7)).item())

    # Средний IoU по всем классам
    valid_ious = [iou for iou in ious if not np.isnan(iou)]
    return np.mean(valid_ious) if valid_ious else 0.0

```

Рисунок 10 – Метрика IoU

```

def train_and_evaluate(name, filters_base, kernel_size, activation, epochs=7):
    print(f"\n{' '*60}")
    print(f"Эксперимент: {name} (f={filters_base}, k={kernel_size}, act={activation})")
    print(f"{' '*60}")

    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    model = create_unet(n_classes=NUM_CLASSES, filters_base=filters_base,
                        kernel_size=kernel_size, activation=activation)
    model = model.to(device)

    optimizer = optim.Adam(model.parameters(), lr=1e-4)
    criterion = nn.CrossEntropyLoss()

    history = {
        'train_loss': [], 'train_acc': [], 'train_iou': [],
        'val_loss': [], 'val_acc': [], 'val_iou': []
    }

    for epoch in range(epochs):
        # Обучение
        model.train()
        train_loss = 0
        train_correct = 0
        train_total = 0
        train_iou_sum = 0

        for images, masks in train_loader:
            images, masks = images.to(device), masks.to(device)

            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, masks)
            loss.backward()
            optimizer.step()

            train_loss += loss.item()
            preds = torch.argmax(outputs, dim=1)
            train_correct += (preds == masks).sum().item()
            train_total += masks.numel()
            train_iou_sum += iou_score(outputs, masks, NUM_CLASSES)

```

Рисунок 11 – Функция обучения


```

# Валидация
model.eval()
val_loss = 0
val_correct = 0
val_total = 0
val_iou_sum = 0

with torch.no_grad():
    for images, masks in val_loader:
        images, masks = images.to(device), masks.to(device)
        outputs = model(images)
        loss = criterion(outputs, masks)

        val_loss += loss.item()
        preds = torch.argmax(outputs, dim=1)
        val_correct += (preds == masks).sum().item()
        val_total += masks.numel()
        val_iou_sum += iou_score(outputs, masks, NUM_CLASSES)

train_loss_avg = train_loss / len(train_loader)
train_acc = train_correct / train_total
train_iou = train_iou_sum / len(train_loader)

val_loss_avg = val_loss / len(val_loader)
val_acc = val_correct / val_total
val_iou = val_iou_sum / len(val_loader)

history['train_loss'].append(train_loss_avg)
history['train_acc'].append(train_acc)
history['train_iou'].append(train_iou)
history['val_loss'].append(val_loss_avg)
history['val_acc'].append(val_acc)
history['val_iou'].append(val_iou)

print(f"Epoch {epoch+1}/{epochs} - "
      f"loss: {train_loss_avg:.4f}, acc: {train_acc:.4f}, iou: {train_iou:.4f} - "
      f"val_loss: {val_loss_avg:.4f}, val_acc: {val_acc:.4f}, val_iou: {val_iou:.4f}")

result = {
    'name': name,
    'filters': filters_base,
    'kernel': kernel_size,
    'activation': activation,
    'history': history
}

```

Рисунок 12 – Функция обучения

```

experiments = [
    # Изменение фильтров
    ("Filters_16", 16, 3, "relu"),
    ("Base_24x3_relu", 24, 3, "relu"),
    ("Filters_32", 32, 3, "relu"),

    # Изменение kernel_size
    ("Kernel_5", 24, 5, "relu"),
    ("Kernel_7", 24, 7, "relu"),

    # Изменение активации
    ("Act_linear", 24, 3, "linear"),
    ("Act_selu", 24, 3, "selu"),
    ("Act_elu", 24, 3, "elu"),

    # Комбинированные (filters=24 вместо 32 для снижения нагрузки)
    ("Comb_F16_K5_elu", 16, 5, "elu"),
    ("Comb_F24_K7_selu", 24, 7, "selu"),
]
EPOCHS = 7

```

Рисунок 13 – Список экспериментов

```

all_results = []

for name, f, k, act in experiments:
    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()

    result = train_and_evaluate(name, f, k, act, EPOCHS)
    all_results.append(result)

    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()

    print(f"\nЭксперимент '{name}' завершен\n")
    print("="*60)

```

Рисунок 14 – Обучение всех экспериментов

Эксперимент	Val Acc	Val IoU
Filters_16	0.7103	0.3344
Base_24x3_relu	0.7279	0.3717
Filters_32	0.7261	0.3649
Kernel_5	0.7157	0.3343
Kernel_7	0.7330	0.3618
Act_linear	0.6765	0.3089
Act_selu	0.7243	0.3371
Act_elu	0.7291	0.3594
Comb_F16_K5_elu	0.7402	0.3510
Comb_F24_K7_selu	0.7336	0.3522

Лучшая: Base_24x3_relu (Val IoU = 0.3717)

Рисунок 15 – Итоговая таблица результатов



Рисунок 16 – Визуализация лучшей модели

Tensorflow:

Задание 1. На основе учебного ноутбука проведите финальную подготовку данных. Измените количество сегментирующих классов с 16 на 5. Проведите суммарно не менее 10 экспериментов и визуализируйте их результаты (включая точность обучения сетей на одинаковом количестве эпох, например, на 7), изменив:

- количество фильтров в сверточных слоях
- размер ядра в сверточных слоях
- функцию активации в скрытых слоях

```
# Импортируем модели keras: Model
from tensorflow.keras.models import Model
import tensorflow as tf

# Импортируем стандартные слои keras
from tensorflow.keras.layers import Input, Conv2DTranspose, concatenate, Activation
from tensorflow.keras.layers import MaxPooling2D, Conv2D, BatchNormalization, UpSampling2D

# Импортируем оптимизатор Adam
from tensorflow.keras.optimizers import Adam

# Импортируем модуль pyplot библиотеки matplotlib для построения графиков
import matplotlib.pyplot as plt

# Импортируем модуль image для работы с изображениями
from tensorflow.keras.preprocessing import image

# Импортируем библиотеку numpy
import numpy as np

# Импортируем метод деления выборки
from sklearn.model_selection import train_test_split

# загрузка файлов по HTML ссылке
import gdown

# Для работы с файлами
import os

# Для генерации случайных чисел
import random

import time

# импортируем модель Image для работы с изображениями
from PIL import Image

# очистка ОЗУ
import gc
```

Рисунок 17 – Импорт библиотек

```
# Загрузка датасета из облака

gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/l14/construction_256x192.zip', None, quiet=False)
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/l14/construction_512x384.zip', None, quiet=False)

!unzip -q 'construction_256x192.zip' # распаковываем архив
```

Рисунок 18 – Импорт библиотек

```
# Глобальные параметры

IMG_WIDTH = 192          # Ширина картинки
IMG_HEIGHT = 256         # Высота картинки
NUM_CLASSES = 16         # Задаем количество классов на изображении
TRAIN_DIRECTORY = 'train' # Название папки с файлами обучающей выборки
VAL_DIRECTORY = 'val'     # Название папки с файлами проверочной выборки
```

Рисунок 19 – Глобальные параметры

```
train_images = [] # Создаем пустой список для хранения оригинальных изображений обучающей выборки
val_images = [] # Создаем пустой список для хранения оригинальных изображений проверочной выборки

cur_time = time.time() # Засекаем текущее время

# Проходим по всем файлам в каталоге по указанному пути
for filename in sorted(os.listdir(TRAIN_DIRECTORY+'/original')):
    # Читаем очередную картинку и добавляем ее в список изображений с указанным target_size
    train_images.append(image.load_img(os.path.join(TRAIN_DIRECTORY+'/original',filename),
                                                target_size=(IMG_WIDTH, IMG_HEIGHT)))

# Отображаем время загрузки картинок обучающей выборки
print ('Обучающая выборка загружена. Время загрузки: ', round(time.time() - cur_time, 2), 'с', sep='')

# Отображаем количество элементов в обучающей выборке
print ('Количество изображений: ', len(train_images))

cur_time = time.time() # Засекаем текущее время

# Проходим по всем файлам в каталоге по указанному пути
for filename in sorted(os.listdir(VAL_DIRECTORY+'/original')):
    # Читаем очередную картинку и добавляем ее в список изображений с указанным target_size
    val_images.append(image.load_img(os.path.join(VAL_DIRECTORY+'/original',filename),
                                                target_size=(IMG_WIDTH, IMG_HEIGHT)))

# Отображаем время загрузки картинок проверочной выборки
print ('Проверочная выборка загружена. Время загрузки: ', round(time.time() - cur_time, 2), 'с', sep='')

# Отображаем количество элементов в проверочной выборке
print ('Количество изображений: ', len(val_images))
```

Рисунок 20 – Загрузка оригинальных изображений

```

train_segments = [] # Создаем пустой список для хранения оригинальных изображений обучающей выборки
val_segments = [] # Создаем пустой список для хранения оригинальных изображений проверочной выборки

cur_time = time.time() # Засекаем текущее время

for filename in sorted(os.listdir(TRAIN_DIRECTORY+'/segment')): # Проходим по всем файлам в каталоге по указанному пути
    # Читаем очередную картинку и добавляем ее в список изображений с указанным target_size
    train_segments.append(image.load_img(os.path.join(TRAIN_DIRECTORY+'/segment',filename),
                                                target_size=(IMG_WIDTH, IMG_HEIGHT)))

# Отображаем время загрузки картинок обучающей выборки
print ('Обучающая выборка загружена. Время загрузки: ', round(time.time() - cur_time, 2), 'с', sep='')

# Отображаем количество элементов в обучающем наборе сегментированных изображений
print ('Количество изображений: ', len(train_segments))

cur_time = time.time() # Засекаем текущее время

for filename in sorted(os.listdir(VAL_DIRECTORY+'/segment')): # Проходим по всем файлам в каталоге по указанному пути
    # Читаем очередную картинку и добавляем ее в список изображений с указанным target_size
    val_segments.append(image.load_img(os.path.join(VAL_DIRECTORY+'/segment',filename),
                                                target_size=(IMG_WIDTH, IMG_HEIGHT)))

# Отображаем время загрузки картинок проверочной выборки
print ('Проверочная выборка загружена. Время загрузки: ', round(time.time() - cur_time, 2), 'с', sep='')

# Отображаем количество элементов в проверочном наборе сегментированных изображений
print ('Количество изображений: ', len(val_segments))

```

Рисунок 21 – Загрузка сегментированных изображений

```

# Определяем цветовую карту для 5 классов
CLASS_COLORS_5 = [
    (0, 0, 0),
    (100, 0, 0),
    (0, 100, 0),
    (0, 0, 100),
    (100, 100, 0)
]
NUM_CLASSES = 5

```

Рисунок 22 – Определение цветовой карты для 5 классов

```

def segments_to_array_5(img_list, class_colors, num_classes=5):
    num_samples = len(img_list)
    h, w = img_list[0].size[1], img_list[0].size[0]
    result = np.zeros((num_samples, h, w, num_classes), dtype='float32')
    for i, img in enumerate(img_list):
        img_arr = np.array(img)
        for class_idx, color in enumerate(class_colors):
            mask = np.all(img_arr == color, axis=-1)
            result[i, :, :, class_idx] = mask.astype('float32')
    bg_mask = 1.0 - np.sum(result, axis=-1)
    result[:, :, :, 0] = np.maximum(result[:, :, :, 0], bg_mask)
    return result

```

Рисунок 23 – Конвертация масок в one-hot

```

# X храним как uint8
x_train = np.array([image.img_to_array(img) for img in train_images], dtype='uint8')
x_val = np.array([image.img_to_array(img) for img in val_images], dtype='uint8')

# Y храним как int8
def segments_to_class_map(seg_list, class_colors, num_classes=5):
    h, w = seg_list[0].size[1], seg_list[0].size[0]
    result = np.zeros((len(seg_list), h, w), dtype='int8')

    for i, seg in enumerate(seg_list):
        seg_arr = np.array(seg)
        for class_idx, color in enumerate(class_colors):
            mask = np.all(seg_arr == color, axis=-1)
            result[i, mask] = class_idx
    return result

y_train = segments_to_class_map(train_segments, CLASS_COLORS_5, NUM_CLASSES)
y_val = segments_to_class_map(val_segments, CLASS_COLORS_5, NUM_CLASSES)

print(f"x_train: {x_train.shape} dtype={x_train.dtype} ~{x_train.nbytes/1024/1024:.1f} MB")
print(f"y_train: {y_train.shape} dtype={y_train.dtype} ~{y_train.nbytes/1024/1024:.1f} MB")

```

Рисунок 24 – Подготовка данных

```

def build_unet_custom(input_shape, num_classes, filters_base=24, kernel_size=3, activation='relu'):
    inputs = Input(input_shape)

    x = Rescaling(1.0/255.0)(inputs)

    # Энкодер
    c1 = Conv2D(filters_base, kernel_size, padding='same')(x)
    c1 = BatchNormalization()(c1)
    c1 = Activation(activation)(c1)
    c1 = Conv2D(filters_base, kernel_size, padding='same')(c1)
    c1 = BatchNormalization()(c1)
    c1 = Activation(activation)(c1)
    p1 = MaxPooling2D(2)(c1)

    c2 = Conv2D(filters_base*2, kernel_size, padding='same')(p1)
    c2 = BatchNormalization()(c2)
    c2 = Activation(activation)(c2)
    c2 = Conv2D(filters_base*2, kernel_size, padding='same')(c2)
    c2 = BatchNormalization()(c2)
    c2 = Activation(activation)(c2)
    p2 = MaxPooling2D(2)(c2)

    c3 = Conv2D(filters_base*4, kernel_size, padding='same')(p2)
    c3 = BatchNormalization()(c3)
    c3 = Activation(activation)(c3)
    c3 = Conv2D(filters_base*4, kernel_size, padding='same')(c3)
    c3 = BatchNormalization()(c3)
    c3 = Activation(activation)(c3)
    p3 = MaxPooling2D(2)(c3)

    c4 = Conv2D(filters_base*8, kernel_size, padding='same')(p3)
    c4 = BatchNormalization()(c4)
    c4 = Activation(activation)(c4)
    c4 = Conv2D(filters_base*8, kernel_size, padding='same')(c4)
    c4 = BatchNormalization()(c4)
    c4 = Activation(activation)(c4)
    p4 = MaxPooling2D(2)(c4)

    # Bottleneck
    c5 = Conv2D(filters_base*16, kernel_size, padding='same')(p4)
    c5 = BatchNormalization()(c5)
    c5 = Activation(activation)(c5)
    c5 = Conv2D(filters_base*16, kernel_size, padding='same')(c5)
    c5 = BatchNormalization()(c5)
    c5 = Activation(activation)(c5)

```

Рисунок 25 – Архитектура U-net


```

# Декодер
u6 = Conv2DTranspose(filters_base*8, 2, strides=2, padding='same')(c5)
u6 = concatenate([u6, c4])
c6 = Conv2D(filters_base*8, kernel_size, padding='same')(u6)
c6 = BatchNormalization()(c6)
c6 = Activation(activation)(c6)
c6 = Conv2D(filters_base*8, kernel_size, padding='same')(c6)
c6 = BatchNormalization()(c6)
c6 = Activation(activation)(c6)

u7 = Conv2DTranspose(filters_base*4, 2, strides=2, padding='same')(c6)
u7 = concatenate([u7, c3])
c7 = Conv2D(filters_base*4, kernel_size, padding='same')(u7)
c7 = BatchNormalization()(c7)
c7 = Activation(activation)(c7)
c7 = Conv2D(filters_base*4, kernel_size, padding='same')(c7)
c7 = BatchNormalization()(c7)
c7 = Activation(activation)(c7)

u8 = Conv2DTranspose(filters_base*2, 2, strides=2, padding='same')(c7)
u8 = concatenate([u8, c2])
c8 = Conv2D(filters_base*2, kernel_size, padding='same')(u8)
c8 = BatchNormalization()(c8)
c8 = Activation(activation)(c8)
c8 = Conv2D(filters_base*2, kernel_size, padding='same')(c8)
c8 = BatchNormalization()(c8)
c8 = Activation(activation)(c8)

u9 = Conv2DTranspose(filters_base, 2, strides=2, padding='same')(c8)
u9 = concatenate([u9, c1])
c9 = Conv2D(filters_base, kernel_size, padding='same')(u9)
c9 = BatchNormalization()(c9)
c9 = Activation(activation)(c9)
c9 = Conv2D(filters_base, kernel_size, padding='same')(c9)
c9 = BatchNormalization()(c9)
c9 = Activation(activation)(c9)

outputs = Conv2D(num_classes, 1, activation='softmax')(c9)
return Model(inputs=[inputs], outputs=[outputs])

```

Рисунок 26 – Архитектура U-net

```

def iou_coef(y_true, y_pred):
    y_true = tf.cast(y_true, tf.int32)
    y_pred = tf.argmax(y_pred, axis=-1)
    y_pred = tf.cast(y_pred, tf.int32)
    intersection = tf.reduce_sum(tf.cast(tf.equal(y_true, y_pred), tf.float32))
    union = tf.reduce_sum(tf.cast(tf.not_equal(y_true, y_pred), tf.float32)) + intersection
    return intersection / (union + 1e-7)

```

Рисунок 27 – Метрика IoU

```
def train_and_evaluate(name, filters_base, kernel_size, activation, epochs=7):
    print(f"\n Эксперимент: {name} (f={filters_base}, k={kernel_size}, act={activation})")

    model = build_unet_custom((IMG_WIDTH, IMG_HEIGHT, 3), NUM_CLASSES, filters_base, kernel_size, activation)
    model.compile(optimizer=Adam(1e-4), loss='sparse_categorical_crossentropy', metrics=['accuracy', iou_coef])

    train_dataset = tf.data.Dataset.from_tensor_slices((x_train, y_train))
    train_dataset = train_dataset.shuffle(256).batch(16).prefetch(tf.data.AUTOTUNE)

    val_dataset = tf.data.Dataset.from_tensor_slices((x_val, y_val))
    val_dataset = val_dataset.batch(16).prefetch(tf.data.AUTOTUNE)

    history = model.fit(
        train_dataset,
        epochs=epochs,
        validation_data=val_dataset,
        verbose=1
    )

    result = {
        'name': name,
        'filters': filters_base,
        'kernel': kernel_size,
        'activation': activation,
        'history': history.history,
        'model': model
    }
    return result
```

Рисунок 28 – Функция обучения и оценки модели

```
def show_predictions(results, x_val, y_val, colors, n=2):
    for r in results[:3]:
        print(f"\n--- {r['name']} ---")
        preds = r['model'].predict(x_val[:n])
        for i in range(n):
            plt.figure(figsize=(16, 4))
            plt.subplot(1, 4, 1); plt.imshow(x_val[i]); plt.title('Оригинал'); plt.axis('off')
            plt.subplot(1, 4, 2); plt.imshow(mask_to_rgb(y_val[i], colors)); plt.title('Истина'); plt.axis('off')
            plt.subplot(1, 4, 3); plt.imshow(mask_to_rgb(preds[i], colors)); plt.title('Предсказание'); plt.axis('off')
            diff = (np.argmax(y_val[i], axis=-1) != np.argmax(preds[i], axis=-1)).astype(np.uint8) * 255
            plt.subplot(1, 4, 4); plt.imshow(diff, cmap='gray'); plt.title('Ошибки'); plt.axis('off')
            plt.show()
```

Рисунок 29 – Функция визуализации предсказаний

```

experiments = [
    # Изменение фильтров (3 эксперимента)
    ("Filters_16", 16, 3, "relu"),
    ("Base_24x3_relu", 24, 3, "relu"),
    ("Filters_32", 32, 3, "relu"),

    # Изменение kernel_size (2 эксперимента)
    ("Kernel_5", 24, 5, "relu"),
    ("Kernel_7", 24, 7, "relu"),

    # Изменение активации (3 эксперимента)
    ("Act_linear", 24, 3, "linear"),
    ("Act_selu", 24, 3, "selu"),
    ("Act_elu", 24, 3, "elu"),

    # Комбинированные (2 эксперимента)
    ("Comb_F16_K5_elu", 16, 5, "elu"),
    ("Comb_F32_K7_selu", 32, 7, "selu"),
]
EPOCHS = 7

```

Рисунок 30 – Список экспериментов

```

all_results = []

for name, f, k, act in experiments:
    gc.collect()
    tf.keras.backend.clear_session()

    result = train_and_evaluate(name, f, k, act, EPOCHS)
    all_results.append(result)

    gc.collect()
    tf.keras.backend.clear_session()

    print(f"\n Эксперимент '{name}' завершен\n")
    print("="*60)

```

Рисунок 31 – Обучение всех экспериментов

Эксперимент	Val Acc	Val IoU
Filters_16	0.6886	0.6822
Base_24x3_relu	0.7121	0.7099
Filters_32	0.7257	0.7233
Kernel_5	0.7276	0.7228
Kernel_7	0.7083	0.7044
Act_linear	0.6656	0.6576
Act_selu	0.7055	0.7001
Act_elu	0.6827	0.6775
Comb_F16_K5_elu	0.7091	0.7008
Comb_F32_K7_selu	0.7072	0.7003

Лучшая: Filters_32 (Val IoU = 0.7233)

Рисунок 32 – Итоговая таблица результатов

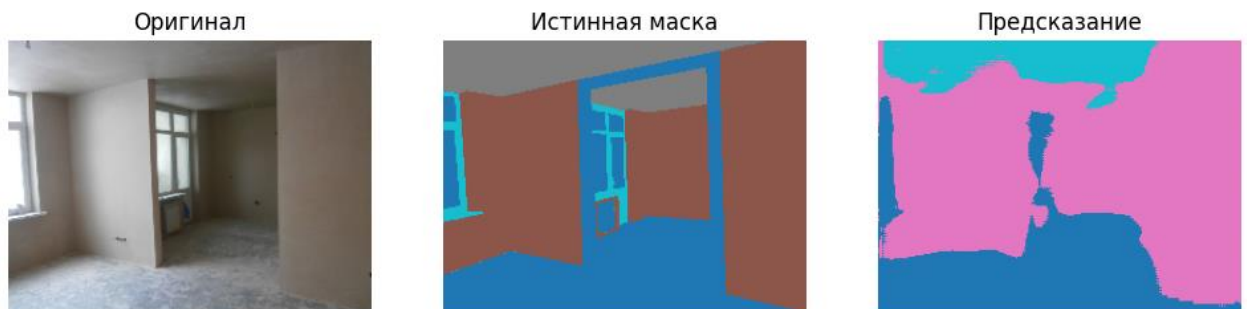


Рисунок 33 – Визуализация лучшей модели

Задание 2. На основе учебного ноутбука проведите финальную подготовку данных. Измените количество сегментирующих классов с 16 на 7. Изучите внимательно особенности U-Net, определите принципиальное отличие U-Net и simpleUnet из учебного ноутбука. Доработайте simpleUnet с учетом особенностей U-Net. Обучите модель на 100 эпохах и визуализируйте результат.

```
# Импортируем модели keras: Model
from tensorflow.keras.models import Model

import tensorflow as tf
# Импортируем стандартные слои keras
from tensorflow.keras.layers import Input, Conv2DTranspose, concatenate, Activation
from tensorflow.keras.layers import MaxPooling2D, Conv2D, BatchNormalization, UpSampling2D

# Импортируем оптимизатор Adam
from tensorflow.keras.optimizers import Adam

# Импортируем модуль pyplot библиотеки matplotlib для построения графиков
import matplotlib.pyplot as plt

# Импортируем модуль image для работы с изображениями
from tensorflow.keras.preprocessing import image

# Импортируем библиотеку numpy
import numpy as np

# Импортируем метод деления выборки
from sklearn.model_selection import train_test_split

# загрузка файлов по HTML ссылке
import gdown

# Для работы с файлами
import os

# Для генерации случайных чисел
import random

import time

# импортируем модель Image для работы с изображениями
from PIL import Image

# очистка ОЗУ
import gc
```

Рисунок 34 – Импорт библиотек

```
# Загрузка датасета из облака

gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/l14/construction_256x192.zip', None, quiet=False)
#gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/l14/construction_512x384.zip', None, quiet=False)

!unzip -q 'construction_256x192.zip' # распаковываем архив
```

Рисунок 35 – Загрузка датасета

```
# Глобальные параметры

IMG_WIDTH = 256          # Ширина картинки
IMG_HEIGHT = 192         # Высота картинки
NUM_CLASSES = 16         # Задаем количество классов на изображении
TRAIN_DIRECTORY = 'train' # Название папки с файлами обучающей выборки
VAL_DIRECTORY = 'val'     # Название папки с файлами проверочной выборки
```

Рисунок 36 – Подготовка данных

```
# Цвета исходного датасета (16 классов)
CLASS_COLORS_16 = [
    (0, 0, 0), (100, 0, 0), (0, 100, 0), (100, 100, 0),
    (0, 0, 100), (100, 0, 100), (0, 100, 100), (100, 100, 100),
    (50, 0, 0), (0, 50, 0), (50, 50, 0), (0, 0, 50),
    (50, 0, 50), (0, 50, 50), (50, 50, 50), (200, 200, 200)
]

MAP_16_TO_7 = np.array([
    0, # 0 - FLOOR
    2, # 1 - WALL
    2, # 2 - WALL
    3, # 3 - APERTURE
    4, # 4 - COLUMN
    1, # 5 - CEILING
    5, # 6 - INVENTORY
    6, # 7 - LAMP/WIRE etc
    2, # 8 - WALL
    2, # 9 - WALL
    3, # 10 - DOOR
    3, # 11 - WINDOW
    4, # 12 - RAILINGS
    4, # 13 - LADDER
    6, # 14 - BEAM/EXTERNAL
    6, # 15 - BATTERY/PEOPLE
], dtype=np.int8)

NUM_CLASSES = 7

CLASS_COLORS_7 = [
    (128, 64, 64), # 0 - FLOOR
    (64, 128, 64), # 1 - CEILING
    (64, 64, 128), # 2 - WALL
    (128, 128, 64), # 3 - APERTURE/DOOR/WINDOW
    (128, 64, 128), # 4 - COLUMN/RAILINGS/LADDER
    (64, 128, 128), # 5 - INVENTORY
    (128, 128, 128), # 6 - LAMP/WIRE/BEAM/EXTERNAL/BATTERY/PEOPLE
]
```

Рисунок 37 – Подготовка данных: маппинг 16 классов в 7 классов

```

# Функция конвертации RGB маски в карту 7 классов
def mask_rgb_to_class7(seg_img):
    seg_arr = np.array(seg_img, dtype=np.uint8)
    if seg_arr.shape[-1] == 4:
        seg_arr = seg_arr[..., :3]

    h, w = seg_arr.shape[:2]
    result = np.zeros((h, w), dtype=np.int8)

    for old_class, color in enumerate(CLASS_COLORS_16):
        mask = np.all(seg_arr == color, axis=-1)
        result[mask] = MAP_16_TO_7[old_class]

    return result

x_train = np.array([np.asarray(img, dtype=np.uint8) for img in train_images])
x_val = np.array([np.asarray(img, dtype=np.uint8) for img in val_images])

print(f'x_train: {x_train.shape} dtype={x_train.dtype} ~{x_train.nbytes/1024/1024:.1f} MB')
h, w = np.array(train_segments[0]).shape[:2]

y_train = np.zeros((len(train_segments), h, w), dtype=np.int8)
y_val = np.zeros((len(val_segments), h, w), dtype=np.int8)

for i, seg in enumerate(train_segments):
    y_train[i] = mask_rgb_to_class7(seg)

for i, seg in enumerate(val_segments):
    y_val[i] = mask_rgb_to_class7(seg)

print(f'y_train: {y_train.shape} dtype={y_train.dtype} ~{y_train.nbytes/1024/1024:.1f} MB')
print(f'Уникальные классы в y_train: {np.unique(y_train)}')

del train_images, val_images, train_segments, val_segments
gc.collect()

```

Рисунок 38 – Подготовка данных: маппинг 16 классов в 7 классов

```

def iou_coef(y_true, y_pred):
    y_true = tf.cast(y_true, tf.int32)
    y_pred = tf.argmax(y_pred, axis=-1)
    y_pred = tf.cast(y_pred, tf.int32)
    intersection = tf.reduce_sum(tf.cast(tf.equal(y_true, y_pred), tf.float32))
    union = tf.reduce_sum(tf.cast(tf.not_equal(y_true, y_pred), tf.float32)) + intersection
    return intersection / (union + 1e-7)

```

Рисунок 39 – Функция IoU

```

def build_unet_full(input_shape, num_classes=7, filters_base=32):
    inputs = Input(input_shape, dtype='uint8')
    x = Rescaling(1.0/255.0)(inputs)

    # Энкодер
    c1 = Conv2D(filters_base, 3, padding='same')(x)
    c1 = BatchNormalization()(c1)
    c1 = Activation('relu')(c1)
    c1 = Conv2D(filters_base, 3, padding='same')(c1)
    c1 = BatchNormalization()(c1)
    c1 = Activation('relu')(c1)
    p1 = MaxPooling2D(2)(c1)

    c2 = Conv2D(filters_base*2, 3, padding='same')(p1)
    c2 = BatchNormalization()(c2)
    c2 = Activation('relu')(c2)
    c2 = Conv2D(filters_base*2, 3, padding='same')(c2)
    c2 = BatchNormalization()(c2)
    c2 = Activation('relu')(c2)
    p2 = MaxPooling2D(2)(c2)

    c3 = Conv2D(filters_base*4, 3, padding='same')(p2)
    c3 = BatchNormalization()(c3)
    c3 = Activation('relu')(c3)
    c3 = Conv2D(filters_base*4, 3, padding='same')(c3)
    c3 = BatchNormalization()(c3)
    c3 = Activation('relu')(c3)
    p3 = MaxPooling2D(2)(c3)

    c4 = Conv2D(filters_base*8, 3, padding='same')(p3)
    c4 = BatchNormalization()(c4)
    c4 = Activation('relu')(c4)
    c4 = Conv2D(filters_base*8, 3, padding='same')(c4)
    c4 = BatchNormalization()(c4)
    c4 = Activation('relu')(c4)

    # Декодер с SKIP CONNECTIONS
    u5 = Conv2DTranspose(filters_base*4, 2, strides=2, padding='same')(c4)
    u5 = concatenate([u5, c3])
    d5 = Conv2D(filters_base*4, 3, padding='same')(u5)
    d5 = BatchNormalization()(d5)
    d5 = Activation('relu')(d5)
    d5 = Conv2D(filters_base*4, 3, padding='same')(d5)
    d5 = BatchNormalization()(d5)
    d5 = Activation('relu')(d5)

```

Рисунок 40 – Построение U-net


```

u6 = Conv2DTranspose(filters_base*2, 2, strides=2, padding='same')(d5)
u6 = concatenate([u6, c2])
d6 = Conv2D(filters_base*2, 3, padding='same')(u6)
d6 = BatchNormalization()(d6)
d6 = Activation('relu')(d6)
d6 = Conv2D(filters_base*2, 3, padding='same')(d6)
d6 = BatchNormalization()(d6)
d6 = Activation('relu')(d6)

u7 = Conv2DTranspose(filters_base, 2, strides=2, padding='same')(d6)
u7 = concatenate([u7, c1])
d7 = Conv2D(filters_base, 3, padding='same')(u7)
d7 = BatchNormalization()(d7)
d7 = Activation('relu')(d7)
d7 = Conv2D(filters_base, 3, padding='same')(d7)
d7 = BatchNormalization()(d7)
d7 = Activation('relu')(d7)

outputs = Conv2D(num_classes, 1, activation='softmax')(d7)

return Model(inputs=inputs, outputs=outputs)

```

Рисунок 41 – Построение U-net

```

EPOCHS = 100
BATCH_SIZE = 32

train_ds = tf.data.Dataset.from_tensor_slices((x_train, y_train))
train_ds = train_ds.shuffle(512).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

val_ds = tf.data.Dataset.from_tensor_slices((x_val, y_val))
val_ds = val_ds.batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

model = build_unet_full((x_train.shape[1], x_train.shape[2], 3), num_classes=NUM_CLASSES, filters_base=32)
model.compile(optimizer=Adam(1e-3), loss='sparse_categorical_crossentropy', metrics=['accuracy', iou_coef])

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS,
    verbose=1,
    callbacks=[
        tf.keras.callbacks.ReduceLROnPlateau(patience=10, factor=0.5),
        tf.keras.callbacks.EarlyStopping(patience=20, restore_best_weights=True)
    ]
)

```

Рисунок 42 – Обучение модели на 100 эпохах

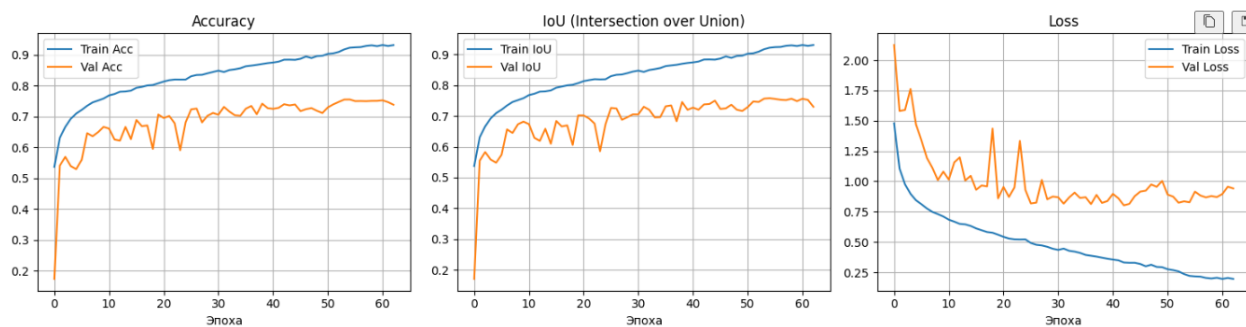


Рисунок 43 – Визуализация графиков обучения

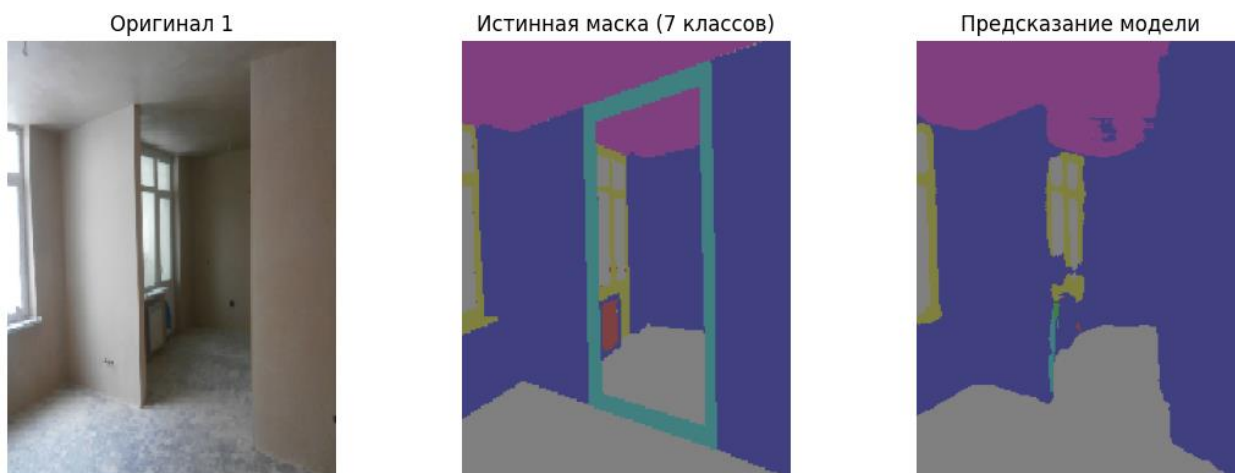


Рисунок 44 – Визуализация предсказаний

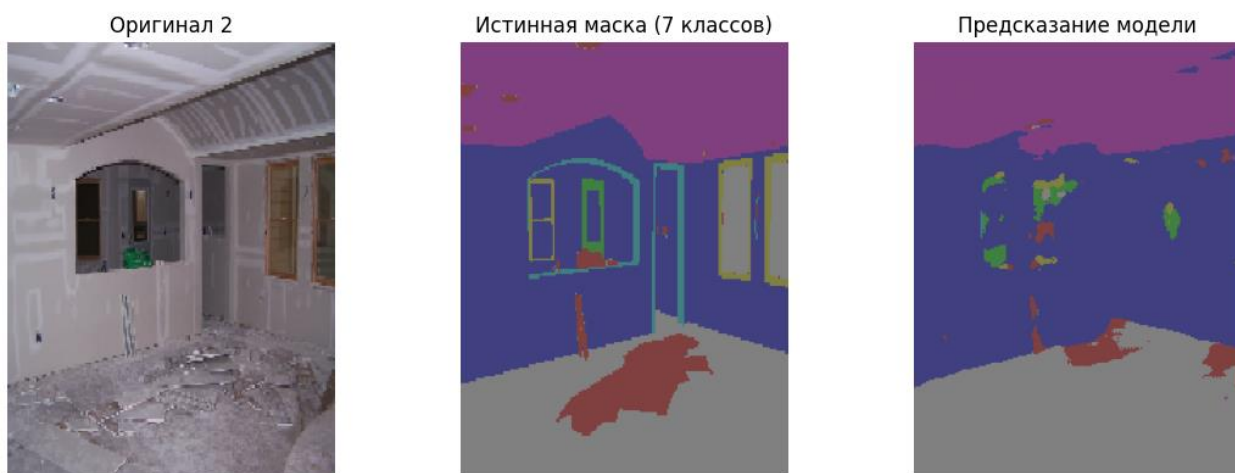


Рисунок 45 – Визуализация предсказаний



Рисунок 46 – Визуализация предсказаний

Пример 1: $\text{IoU} = 0.8782$
 Пример 2: $\text{IoU} = 0.8139$
 Пример 3: $\text{IoU} = 0.7648$

Рисунок 47 – IoU

Задание 3. На основе учебного ноутбука проведите финальную подготовку данных. Измените количество сегментирующих классов с 16 на 7. Реализуйте сегментацию базы Стройка на основе модели PSPNet, изучив статью Pyramid Scene Parsing Network.

```
# Импортируем модели keras: Model
from tensorflow.keras.models import Model

import tensorflow as tf

# Импортируем стандартные слои keras
from tensorflow.keras.layers import Input, Conv2DTranspose, concatenate, Activation
from tensorflow.keras.layers import MaxPooling2D, Conv2D, BatchNormalization, UpSampling2D

# Импортируем оптимизатор Adam
from tensorflow.keras.optimizers import Adam

# Импортируем модуль pyplot библиотеки matplotlib для построения графиков
import matplotlib.pyplot as plt

# Импортируем модуль image для работы с изображениями
from tensorflow.keras.preprocessing import image

# Импортируем библиотеку numpy
import numpy as np

# Импортируем метод деления выборки
from sklearn.model_selection import train_test_split

# загрузка файлов по HTML ссылке
import gdown

# Для работы с файлами
import os

# Для генерации случайных чисел
import random

import time

# импортируем модель Image для работы с изображениями
from PIL import Image

# очистка ОЗУ
import gc
```

Рисунок 48 – Импорт библиотек

```
# Загрузка датасета из облака

gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/l14/construction_256x192.zip', None, quiet=False)
#gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/l14/construction_512x384.zip', None, quiet=False)

!unzip -q 'construction_256x192.zip' # распаковываем архив
```

Рисунок 49 – Загрузка датасета

```

# Глобальные параметры
IMG_WIDTH = 256
IMG_HEIGHT = 192
NUM_CLASSES = 16
TRAIN_DIRECTORY = 'train'
VAL_DIRECTORY = 'val'

# Загрузка оригинальных изображений
train_images = []
val_images = []

print("Загрузка оригинальных изображений...")
for filename in sorted(os.listdir(TRAIN_DIRECTORY+'original')):
    train_images.append(image.load_img(os.path.join(TRAIN_DIRECTORY+'original', filename),
                                                target_size=(IMG_WIDTH, IMG_HEIGHT)))
for filename in sorted(os.listdir(VAL_DIRECTORY+'original')):
    val_images.append(image.load_img(os.path.join(VAL_DIRECTORY+'original', filename),
                                                target_size=(IMG_WIDTH, IMG_HEIGHT)))

print(f"Train images: {len(train_images)}, Val images: {len(val_images)}")

# Загрузка сегментированных изображений
train_segments = []
val_segments = []

print("Загрузка масок...")
for filename in sorted(os.listdir(TRAIN_DIRECTORY+'segment')):
    train_segments.append(image.load_img(os.path.join(TRAIN_DIRECTORY+'segment', filename),
                                                target_size=(IMG_WIDTH, IMG_HEIGHT)))
for filename in sorted(os.listdir(VAL_DIRECTORY+'segment')):
    val_segments.append(image.load_img(os.path.join(VAL_DIRECTORY+'segment', filename),
                                                target_size=(IMG_WIDTH, IMG_HEIGHT)))

print(f"Train segments: {len(train_segments)}, Val segments: {len(val_segments)}")

```

Рисунок 50 – Глобальные параметры и загрузка данных

```

# Цвета исходного датасета (16 классов)
CLASS_COLORS_16 = [
    (0, 0, 0), (100, 0, 0), (0, 100, 0), (100, 100, 0),
    (0, 0, 100), (100, 0, 100), (0, 100, 100), (100, 100, 100),
    (50, 0, 0), (0, 50, 0), (50, 50, 0), (0, 0, 50),
    (50, 0, 50), (0, 50, 50), (50, 50, 50), (200, 200, 200)
]

# Маппинг 16 -> 7 классов
# 0=FLOOR, 1=CEILING, 2=WALL, 3=APERTURE, 4=SUPPORT, 5=INVENTORY, 6=OTHER
MAP_16_TO_7 = np.array([
    0, # 0 - FLOOR
    2, # 1 - WALL
    2, # 2 - WALL
    3, # 3 - APERTURE
    4, # 4 - COLUMN/SUPPORT
    1, # 5 - CEILING
    5, # 6 - INVENTORY
    6, # 7 - LAMP/WIRE/OTHER
    2, # 8 - WALL
    2, # 9 - WALL
    3, # 10 - DOOR
    3, # 11 - WINDOW
    4, # 12 - RAILINGS
    4, # 13 - LADDER
    6, # 14 - BEAM/OTHER
    6, # 15 - BATTERY/PEOPLE
], dtype=np.int8)

NUM_CLASSES = 7

```

Рисунок 51 – Маппинг 16 классов в 7 классов

```

# Функция конвертации RGB маски в карту 7 классов
def build_color_lut(colors):
    lut = np.full(256**3, 255, dtype=np.uint8)
    for idx, (r, g, b) in enumerate(colors):
        key = (r << 16) | (g << 8) | b
        lut[key] = idx
    return lut

LUT_16 = build_color_lut(CLASS_COLORS_16)

def mask_rgb_to_class7(seg_img):
    seg_arr = np.array(seg_img, dtype=np.uint8)
    if seg_arr.shape[-1] == 4:
        seg_arr = seg_arr[..., :3]
    keys = ((seg_arr[...,0].astype(np.uint32) << 16) |
            (seg_arr[...,1].astype(np.uint32) << 8) |
            seg_arr[...,2].astype(np.uint32))
    cls16 = LUT_16[keys]
    cls16 = np.where(cls16 == 255, 0, cls16)
    return MAP_16_TO_7[cls16]

print("Подготовка данных...")

# X храним как uint8
x_train = np.array([np.asarray(img, dtype=np.uint8) for img in train_images])
x_val = np.array([np.asarray(img, dtype=np.uint8) for img in val_images])

print(f"x_train: {x_train.shape} dtype={x_train.dtype} ~{x_train.nbytes/1024/1024:.1f} MB")
print(f"x_val: {x_val.shape} ~{x_val.nbytes/1024/1024:.1f} MB")

# Y храним как int8 карту классов
h, w = np.array(train_segments[0]).shape[:2]
y_train = np.zeros((len(train_segments), h, w), dtype=np.int8)
y_val = np.zeros((len(val_segments), h, w), dtype=np.int8)

for i, seg in enumerate(train_segments):
    y_train[i] = mask_rgb_to_class7(seg)
for i, seg in enumerate(val_segments):
    y_val[i] = mask_rgb_to_class7(seg)

print(f"y_train: {y_train.shape} dtype={y_train.dtype} ~{y_train.nbytes/1024/1024:.1f} MB")
print(f"Уникальные классы: {np.unique(y_train)}")

```

Рисунок 52 – Мappings 16 классов в 7 классов

```

def iou_coef(y_true, y_pred):
    y_true = tf.cast(y_true, tf.int32)
    y_pred = tf.argmax(y_pred, axis=-1)
    y_pred = tf.cast(y_pred, tf.int32)
    intersection = tf.reduce_sum(tf.cast(tf.equal(y_true, y_pred), tf.float32))
    union = tf.reduce_sum(tf.cast(tf.not_equal(y_true, y_pred), tf.float32)) + intersection
    return intersection / (union + 1e-7)

```

Рисунок 53 – Метрика IoU

```

from tensorflow.keras.layers import (Input, Conv2D, BatchNormalization, Activation,
                                     MaxPooling2D, AveragePooling2D, UpSampling2D,
                                     concatenate, Rescaling, Dropout, Resizing)
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam

def conv_bn_relu(x, filters, kernel_size=3, strides=1):
    x = Conv2D(filters, kernel_size, strides=strides, padding='same', use_bias=False)(x)
    x = BatchNormalization()(x)
    x = Activation('relu')(x)
    return x

def backbone(x, filters_base=32):
    # Блок 1
    x = conv_bn_relu(x, filters_base)
    x = conv_bn_relu(x, filters_base)
    x = MaxPooling2D(2)(x) # /2

    # Блок 2
    x = conv_bn_relu(x, filters_base * 2)
    x = conv_bn_relu(x, filters_base * 2)
    x = MaxPooling2D(2)(x) # /4

    # Блок 3
    x = conv_bn_relu(x, filters_base * 4)
    x = conv_bn_relu(x, filters_base * 4)
    x = MaxPooling2D(2)(x) # /8

    # Блок 4
    x = conv_bn_relu(x, filters_base * 8)
    x = conv_bn_relu(x, filters_base * 8)

    return x

```

Рисунок 54 – Архитектура PSPNet

```

def pyramid_pooling_module(x, pool_sizes=(1, 2, 3, 6)):
    h, w = x.shape[1], x.shape[2]
    c = x.shape[-1]
    branches = [x]

    for ps in pool_sizes:
        pool_h = max(h // ps, 1)
        pool_w = max(w // ps, 1)

        pooled = AveragePooling2D(pool_size=(pool_h, pool_w), strides=(pool_h, pool_w))(x)
        pooled = Conv2D(c // 4, 1, padding='same', use_bias=False)(pooled)
        pooled = BatchNormalization()(pooled)
        pooled = Activation('relu')(pooled)
        pooled = UpSampling2D(size=(pool_h, pool_w), interpolation='bilinear')(pooled)
        pooled = Resizing(h, w, interpolation='bilinear')(pooled)
        branches.append(pooled)

    return concatenate(branches, axis=-1)

def build_pspnet(input_shape, num_classes=7, filters_base=32):
    inputs = Input(shape=input_shape, dtype='uint8')
    x = Rescaling(1.0/255.0)(inputs)

    feat = backbone(x, filters_base)

    ppm = pyramid_pooling_module(feat)

    x = conv_bn_relu(ppm, filters_base * 4)
    x = Dropout(0.1)(x)

    x = UpSampling2D(size=(8, 8), interpolation='bilinear')(x)
    x = Resizing(input_shape[0], input_shape[1], interpolation='bilinear')(x)

    x = conv_bn_relu(x, filters_base * 2)
    outputs = Conv2D(num_classes, 1, activation='softmax', padding='same')(x)

    model = Model(inputs=inputs, outputs=outputs, name='PSPNet')
    return model

```

Рисунок 55 – Архитектура PSPNet


```

EPOCHS = 60
BATCH_SIZE = 8

_, H, W, _ = x_train.shape
print(f"Размер входных данных: H={H}, W={W}")

# Строим модель
model = build_pspnet(input_shape=(H, W, 3), num_classes=NUM_CLASSES, filters_base=32)

model.compile(
    optimizer=Adam(learning_rate=1e-3),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy', iou_coef]
)

# Датасеты
train_ds = tf.data.Dataset.from_tensor_slices((x_train, y_train))
train_ds = train_ds.shuffle(512).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

val_ds = tf.data.Dataset.from_tensor_slices((x_val, y_val))
val_ds = val_ds.batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

# Callbacks
callbacks = [
    tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=15, restore_best_weights=True, verbose=1),
    tf.keras.callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=7, min_lr=1e-6, verbose=1),
]

history_pspnet = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS,
    callbacks=callbacks,
    verbose=1
)

print(f"Финальная val_accuracy: {history_pspnet.history['val_accuracy'][-1]:.4f}")
print(f"Финальная val_iou: {history_pspnet.history['val_iou_coef'][-1]:.4f}")

```

Рисунок 56 – Обучение на 60 эпохах

```

Epoch 38: early stopping
Restoring model weights from the end of the best epoch: 23.
Финальная val_accuracy: 0.7657
Финальная val_iou: 0.7661

```

Рисунок 57 – Результат обучения

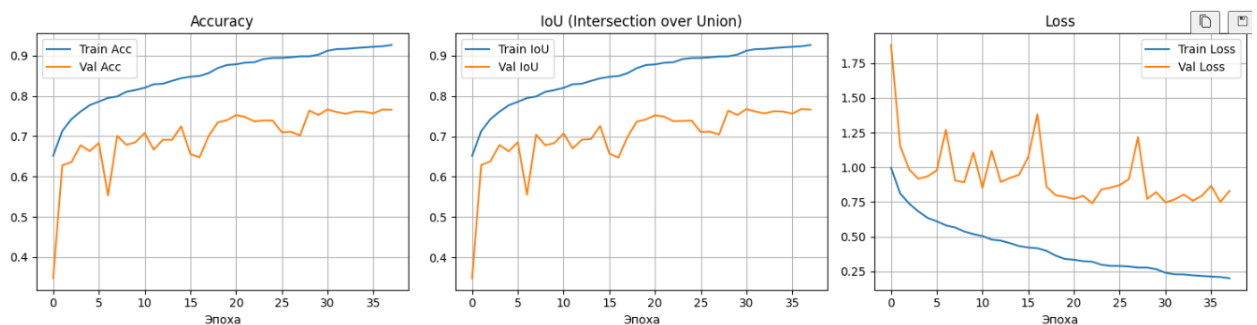


Рисунок 58 – Визуализация графиков обучения



Рисунок 59 – Визуализация предсказаний



Рисунок 60 – Визуализация предсказаний

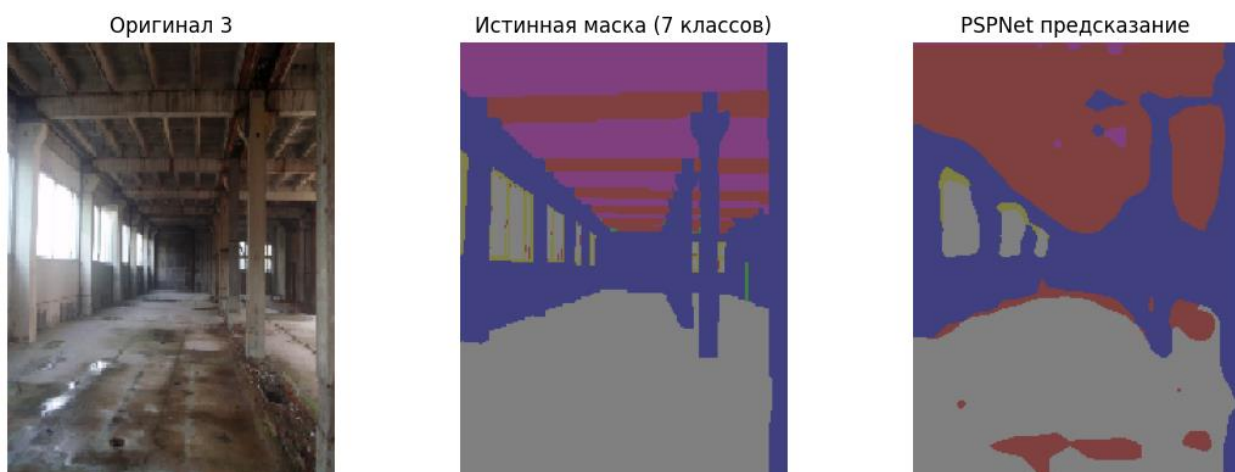


Рисунок 61 – Визуализация предсказаний

IoU на показанных примерах:
 Пример 1: $\text{IoU} = 0.8485$
 Пример 2: $\text{IoU} = 0.7563$
 Пример 3: $\text{IoU} = 0.6561$

Рисунок 62 – IoU

Вывод: в ходе лабораторной работы были изучены современные архитектуры глубоких нейронных сетей для задачи семантической сегментации изображений. В первом задании проведен сравнительный анализ влияния гиперпараметров (количество фильтров, размер ядра, функция активации) на качество сегментации в архитектуре U-Net. Результаты показали, что наилучшая конфигурация (`filters=32`, `kernel_size=3`, `activation=relu`) достигает $\text{Val IoU} = 0.7233$.

Во втором задании реализована полноценная архитектура U-Net с `skip connections`, обученная на 100 эпохах. Ключевое отличие от `simpleUnet` заключается в наличии прямых связей между энкодером и декодером, что позволяет восстанавливать четкие границы объектов. Результат составил $\text{Val IoU} = 0.7300$.

В третьем задании реализована архитектура PSPNet с `Pyramid Pooling Module`, обеспечивающая захват мультимасштабного контекста. Наилучший достигнутый результат $\text{Val IoU} = 0.7661$. Также были освоены методы оптимизации памяти (хранение данных в форматах `uint8` и `int8`, использование `tf.data.Dataset` и `gc.collect`), критически важные для работы с большими наборами данных.